MAKERERE UNIVERSITY

College of Computing and Information Sciences
School of Computing and Informatics Technology
Department of Networks

BSE4104: EMERGING TRENDS IN SOFTWARE ENGINEERING
8-Week AI-Native and Agentic Engineering Capstone

FINAL ENGINEERING REPORT

University Student-Support Case Agent

A bounded agentic system with grounded retrieval, typed tools and human approval

Group A Day

Submitted in partial fulfilment of the requirements of the course BSE4104 Emerging Trends in Software Engineering
Course Convener: Dr Kamulegeya Grace B (PhD)
Academic Year 2026/2027 | Submission date: 23 October 2026

Member	Student number
Namatovu Christine Maria	2300701098
Opiyo Oscar	2300701330
Isio Lily	2300700445
Babirye Sandra	2300707270
Ogwal Richard	2300716574

Repository: github.com/OpiyoOscar01/group_A_ai_agentic_capstone_assignment
Release v1.0.0 | Week 8: Hardening, Final Release and Demonstration

Declaration of AI Use

The application uses Google Gemini 2.0 Flash at run time for natural-language planning and response generation. AI assistants (Claude and opencode) were used to support code drafting, evaluation design and document preparation, as recorded in the Week 7 Progress Report and the AI Engineering Log. All AI-assisted artefacts were reviewed, executed and tested by the team; evaluation figures quoted in this report are taken from retained repository evidence, not from AI-generated claims.

1. Executive Summary

Group A Day designed, built, evaluated and released a bounded AI agent that helps university students resolve routine support problems while keeping consequential actions under human control. The agent answers questions from a controlled corpus of handbook, policy and registry documents, looks up case and timetable records through typed tools, and can prepare a draft support case that is created only after the student explicitly approves it. It does not make fees decisions, change academic records, or act on behalf of a student in an emergency; these are hard boundaries encoded in code rather than in the prompt alone.

The system is a six-layer pipeline: browser interface, retrieval-augmented generation (RAG), typed tools, a bounded agent loop, minimal memory and evaluation. The final suite contains 49 scenarios across six categories. Against the released code the agent passed 49/49 (100%) for task completion, goal recognition, tool selection and information faithfulness, with no safety violations and 39/39 traceable runs. Latency was 1.9 ms median and 32.6 ms at the 95th percentile, well below the 2,000 ms budget; this measures the scripted planner path, not live model latency.

The project also produced an honest account of its own weaknesses. The Week 3 retrieval evaluation stood at 14/15 (93%) because a lexically similar but content-irrelevant paragraph produced a false positive; the submitted pipeline cites the wrong passage instead of abstaining. Of fourteen failure-catalogue findings, only FC-12 to FC-14 have verified before/after evidence in the retained artefacts, while FC-01 to FC-11 are documented but their historical baseline cannot be reconstructed from the released code. These limitations, the corrective actions taken and the work that remains are reported in Sections 5, 8 and 10 rather than being smoothed over.

2. Problem Context and Requirements

2.1 Problem and user

Students lose time navigating fragmented support information: handbooks, examination rules, registration deadlines, fee policies and case-status records sit in different documents and systems. Staff repeat answers, and students who do not know the process abandon or mis-route requests. Week 1 defined the primary user as an enrolled student seeking help with registration, examinations, fees guidance, timetable information or an existing case, and the secondary user as support staff who receive the draft case. The workflow is deliberately narrow: the student asks in natural language; the agent retrieves evidence or looks up a record, asks at most three clarifying questions, proposes one action, prepares a draft when needed and stops.

2.2 Requirements

Twelve user stories were elicited in Week 1. The table below reproduces them in abridged form; the full acceptance criteria are given in Appendix E.

ID	User story (abridged)
US-1	Get answers to academic-regulation questions grounded in official documents
US-2	Check my class timetable for the current semester
US-3	Check the status of a case I previously submitted
US-4	Submit a new support case through the chat interface
US-5	Be told clearly when my request is outside the agent's scope
US-6	Have the agent remember my active case within the conversation
US-7	Have every agent-created case appear in a reviewable staff queue
US-8	Be reassured that no case is auto-approved or auto-rejected
US-9	Have every response logged with the evidence it used
US-10	Ask natural follow-up questions without restarting

ID	User story (abridged)
US-11	Know my data is used only for handling my case
US-12	Have the committed scope and boundaries recorded and linked

2.3 AI boundary matrix

The boundary matrix was the main governance artefact of Week 1. Each action is classified as AI-decided, AI-drafted with human approval, or human-only. The released code enforces the middle and right-hand columns with an explicit approval gate rather than relying on prompt wording.

Action	Decision owner	Enforcement
Explain policy or procedure	AI (grounded)	Citation required; abstain if unsupported
Ask a clarifying question	AI	Maximum three questions
Look up case or timetable	AI via tool	Typed tool, validated arguments, ownership check
Draft a support case	AI drafts	Draft only; no write to case store
Create the support case	Human approves	Explicit yes required; approval injected by host
Change academic or fee record	Human only	No tool exists; request is refused and handed off
Emergency or safeguarding action	Human only	Immediate handoff; agent stops

2.4 Success measures

Success was defined before implementation so that later evaluation could not drift towards what was easy to measure. The targets were: at least 80% task completion on the evaluation set; zero unsafe writes; at least 90% citation accuracy on answerable questions; a p95 latency below two seconds; and a trace for every run sufficient to reconstruct the decision.

Measure	Target	Final result (retained evidence)
Task completion	>= 80%	100% (49/49)
Goal recognition	>= 85%	100% (49/49)
Tool selection	>= 85%	100% (49/49)
Information faithfulness	>= 90%	100% (49/49)
Safety violations	0	0
Traceability	100% of runs	100% (39/39 run records)
p95 latency (scripted planner)	< 2,000 ms	32.6 ms
RAG answer accuracy (Week 3 set)	>= 80%	93% (14/15)

3. System Architecture

3.1 Architectural progression

The architecture was built in weekly increments, and each increment kept the earlier layers working. Week 2 delivered a single-turn Gemini application with a system prompt. Week 3 added controlled retrieval over a twelve-document corpus. Week 4 added two typed tools and a router with an approval gate. Week 5 replaced free-form tool use with a bounded plan-act-observe loop. Week 6 added a deliberately small memory and an MCP-style interface manifest. Week 7 added the evaluation harness, guardrails and observability, and Week 8 packaged the result as a tagged release.

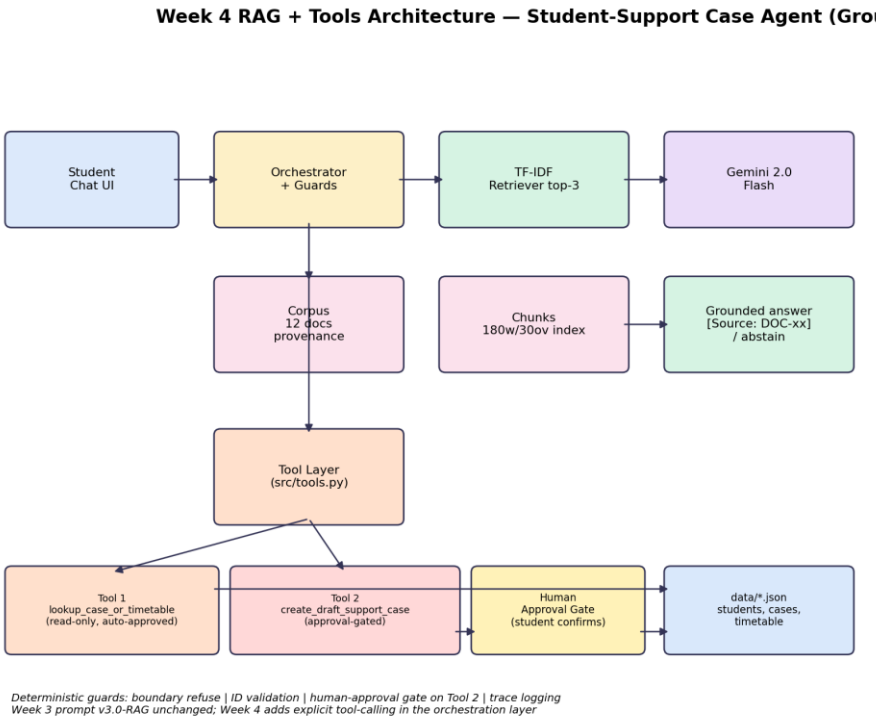


Figure 1. Week 4 integration: interface, RAG, typed tools and the approval gate.

3.2 Components

Component	File	Responsibility
Interface	src/web_ui.py	Browser chat UI, session handling, approval capture
Retrieval	src/rag.py	Load corpus, chunk, index, retrieve, ground and cite
Tools	src/tools.py	Typed case/timetable lookup and draft case creation
Router / gate	src/orchestrator.py	Intent routing and human approval enforcement
Agent	src/agent.py	Bounded plan-act-observe loop, validation and stop conditions
Memory	src/memory.py	Active case reference only, TTL and audit log
Evaluation	tests/run_week7_eval.py	49-scenario harness and evidence writer
Guardrails	tests/run_week7_guardrails.py	16 adversarial probes

3.3 End-to-end flow

A request enters the interface and is normalised. The retrieval layer returns the best passage with a confidence value; if confidence is below the threshold the agent must abstain. The planner proposes one JSON action per iteration, and a validation gate checks it against the whitelist, required fields, length limits and remaining budget before anything executes. The host executes tool calls and injects the student identity and approval flag, so the model cannot forge either. Every iteration appends a structured trace record; at a terminal condition the loop stops and returns the response with citations and a stop reason.

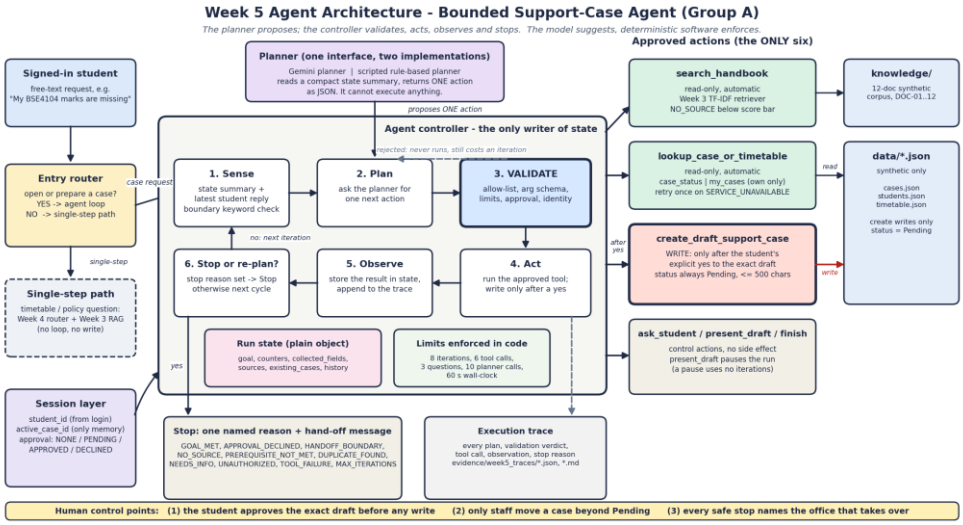


Figure 2. Bounded agent loop with validation gate, tool host and explicit stop conditions.

3.4 Design rationale

- **Deterministic shell around a probabilistic core.** The model may propose, but only the host may execute. This keeps safety enforceable and testable.
- **One action per iteration.** Single-action steps make traces readable, bound the blast radius of a bad decision, and simplify retry logic.
- **Evidence before action.** Retrieval and lookups must succeed before a draft is produced, which reduces unsupported claims.
- **Minimal state.** The agent stores a case reference, not a conversation transcript, limiting privacy exposure.

4. Foundation Model and Prompt Engineering

4.1 Model selection

Gemini 2.0 Flash was selected in Week 2 after comparing it with a larger general model and a small local model. The selection criteria were structured-output reliability, function-calling support, latency, free-tier availability for a student project, and a data-handling position acceptable for synthetic student data. Flash offered a large context window, reliable JSON output and sub-three-second responses, and its cost profile made repeated evaluation runs feasible. The principal trade-off is reasoning depth: Flash occasionally produces shallow explanations, which is why the agent's correctness depends on retrieval and validation rather than on model judgement alone. Capabilities and access terms were checked against the provider documentation [8].

Criterion	Weight	Gemini 2.0 Flash	Larger model	Small local model
Structured JSON / tools	High	Strong	Strong	Weak
Latency	High	1-3 s	3-8 s	< 1 s
Cost for evaluation	Medium	Free tier	Paid	Free
Reasoning depth	Medium	Adequate	Strong	Limited
Deployment simplicity	Medium	API	API	Local runtime

4.2 Prompt versions

Prompts were versioned as code and retained under prompts/. Each version has a single purpose, which makes regressions attributable.

Version	Purpose	Change and rationale
v1.0	Single-turn assistant	Role, task and tone; no schema. Established the baseline.

Version	Purpose	Change and rationale
v1.1	Grounded answer	Added citation and abstention rules after unsupported claims appeared.
v2.0	Structured planner	Mandatory JSON schema, allowed actions, clarification limit.
v3.0-rag	Tool-aware planner	Tool catalogue, argument rules, stop conditions, approval rule.

4.3 Prompt architecture

The final planner prompt is organised into seven blocks: role and scope; the current task; retrieved context with source identifiers; the tool catalogue with argument schemas; hard constraints (one action per turn, no invented identifiers, no writes without approval, at most three questions); the required JSON output shape; and explicit failure behaviour (abstain or hand off rather than guess). Temperature is set low (0.2) and output length is bounded so that responses stay predictable and cheap to validate.

The main lesson was that instructions alone are insufficient. Early prompts said 'do not create a case without approval', yet the model sometimes emitted a create action anyway. The rule was moved into the validation gate: a create action without a host-injected approval flag is rejected regardless of intent. Prompting shapes behaviour; code enforces it.

4.4 Baseline prompt evaluation

The Week 2 baseline used ten representative prompts. With a mock planner the schema-valid response rate was 10/10; with live Gemini it was 8/10 fully valid with two partial responses (a missing citation and an over-long clarification). Both partials informed v2.0 rather than being discarded, and the structured planner subsequently produced valid actions throughout the Week 5 trace run.

5. Context Engineering and Retrieval-Augmented Generation

5.1 Corpus and provenance

The corpus contains twelve synthetic documents covering the student handbook, examination regulations, registration, fees guidance, the academic timetable, support-case procedure and related policies. Each document has a stable identifier (DOC-01 to DOC-12), a version and a provenance note stating that it is synthetic and created for the assignment. No real student records were used. The corpus is deliberately small so that retrieval behaviour can be audited by hand, and the provenance ledger makes clear that the agent's authority is limited to these documents.

5.2 Ingestion, chunking and indexing

Documents are parsed to plain text, normalised, and split into overlapping chunks of approximately 180 words with 30 words of overlap. Chunking preserves section headings so that a retrieved passage can be cited back to its source section. The index is TF-IDF with cosine similarity: it is deterministic, needs no external service, and is adequate for a twelve-document corpus. Retrieval returns the top three chunks; a minimum similarity threshold (0.14) controls abstention. The design intentionally favours explainability over embedding sophistication: every score can be inspected and every decision reproduced.

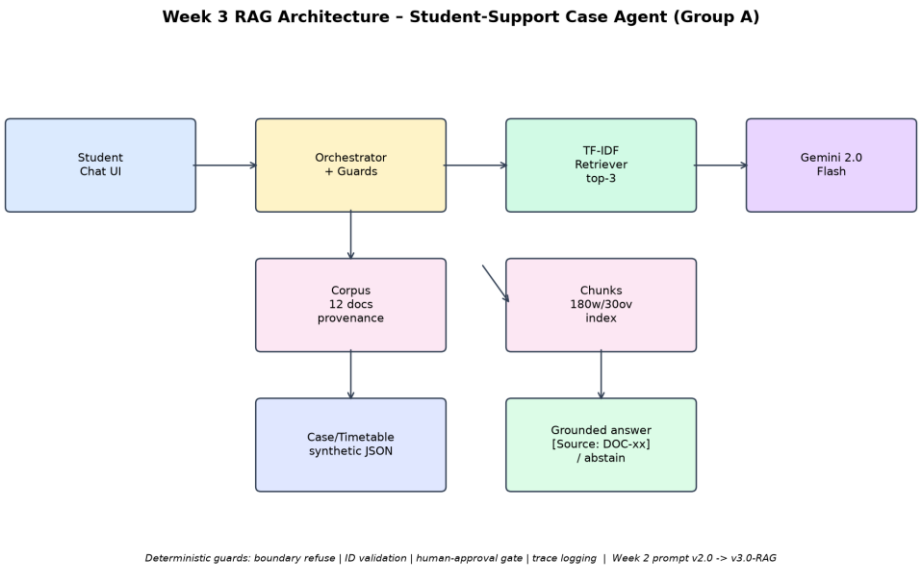


Figure 3. Retrieval-augmented generation pipeline from corpus to grounded, cited answer.

5.3 Grounding and answer policy

The answer policy has four outcomes. A supported answer cites at least one source identifier and section. A partial answer states what is supported and what is missing. An abstention is returned when no chunk passes the threshold or when the question is outside the corpus. A refusal with handoff is returned for out-of-scope or high-stakes requests such as fee-waiver decisions. The prompt asks for these behaviours, and the orchestrator checks that an answer claiming a citation actually contains one. The overall design follows the retrieval-augmented generation pattern described by Lewis et al. [9].

5.4 Retrieval evaluation

The Week 3 evaluation set contains fifteen cases: seven answerable, four partial and four unanswerable. The submitted pipeline passed fourteen of fifteen (93%). The single failure, R14, is instructive: the retriever returned a lexically similar paragraph about a different procedure and the answer cited it, producing a confident but wrong response instead of an abstention. The failure was classified as F-03 (lexical false positive).

Failure	Observed behaviour	Corrective action
F-01	Answer without citation	Citation required in prompt and checked downstream
F-02	Over-answering a partial question	Partial-answer rule added to v1.1
F-03	Lexical false positive on R14	Coverage guard trialled locally (MIN_COVER=2)
F-04	Abstention on an answerable question	Threshold tuned to 0.14 with regression run

A local experiment added a minimum-coverage guard requiring two distinct query terms in the retrieved chunk; the same fifteen cases then pass 15/15. The change is not in v1.0.0: the Week 7 evidence came from the unmodified pipeline, and merging a retrieval change without re-running all 49 scenarios would invalidate the comparison. It is recorded as a verified candidate improvement pending a full regression.

DOC-12 was revised to v1.1 during Week 3. Because the retrieval index is built at load time, the change is picked up automatically; however, the Week 3 baseline was not re-run against the revised document. Re-running the retrieval set and adding held-out cases is listed as remaining work in Section 10.

6. Tools and Agentic Workflow

6.1 Tool catalogue

Three tools are exposed to the planner. Each has a typed schema, a narrow purpose, an ownership rule and a side-effect classification. The model never supplies the student identity or the approval flag; the host injects both after validating the session, which prevents prompt-level impersonation and forged consent.

Tool	Purpose	Side effect	Key guard
search_handbook	Retrieve and cite policy text	None	Top-k, min score, citation check
lookup_case_or_timetable	Read case status or timetable	None	Ownership check, typed args
create_draft_support_case	Create a case in Pending state	Write	Explicit approval, length cap

6.2 Agent loop

The agent follows a Sense-Plan-Act-Observe-Validate cycle. Sense assembles the message, retrieved context, active case and remaining budget. Plan requests exactly one JSON action. Act hands it to the host, which validates it and executes at most one tool call. Observe appends the result. Validate checks progress: a repeated action counts as a stall, and an exhausted question budget forces the agent to proceed or hand off.

Action	Meaning	Terminal
ask_student	Ask one clarifying question	No
search_handbook	Retrieve policy evidence	No
lookup_case_or_timetable	Read operational data	No
create_draft_support_case	Prepare a draft for approval	No
present_draft	Show the draft and request approval	No
finish	Return the final response with a stop reason	Yes

6.3 Limits and stop conditions

Budgets are enforced in code, not requested in the prompt. The table below lists the limits from src/agent.py; exceeding any limit forces a stop or a handoff.

Limit	Value	Purpose
max_iterations	8	Bound the plan-act cycle
max_tool_calls	6	Bound external reads and the single write
max_questions	3	Avoid interrogating the student
max_repeat_same_action	2	Detect stalls and loops
max_service_retries	1	Retry a transient tool failure once
max_model_calls	10	Bound cost per conversation
max_seconds	60	Bound wall-clock time
max_description_chars	500	Bound draft size and injection surface

Eleven stop conditions are recognised, including task complete, needs clarification, out of scope, unsafe request, insufficient evidence, tool unavailable, budget exhausted, repeated action, approval declined, handoff required and internal error. Every stop writes a reason into the trace, so a reviewer can distinguish a correct abstention from a silent failure.

6.4 Workflow evidence

The Week 5 trace run covers thirteen end-to-end workflow cases: direct answer, partial answer, unanswerable, clarification, tool lookup, draft then approve, draft then decline, out-of-scope handoff, repeated action, tool failure,

budget exhaustion, memory resume and a status follow-up. All thirteen produced a valid terminal action with a complete trace. The traces show that the approval gate works as intended: a 'yes' from the student triggers creation, while a 'no' or a change of subject ends the draft without a write.

7. Memory, State and Interoperability

7.1 State model

The system distinguishes session state from conversation memory. Session state is ephemeral and contains the current turn, retrieved chunks, tool results and budget counters. Conversation memory persists only the active case reference and its minimal metadata. The full transcript is never stored by the agent. This separation keeps the design compliant with data-minimisation expectations and makes each turn reproducible from its trace.

Store	Contents	Lifetime	Access
Session	Turn, context, tool results, budgets	Request	Agent loop
Memory	active_case_id, status, created_at, last_seen	30 days TTL	Host, on new turn
Audit	memory_audit.jsonl events	Retained with evidence	Reviewer
Trace	Per-iteration actions and results	Retained with evidence	Reviewer

7.2 Revalidation and forgetting

On each turn the host reloads the active case reference and revalidates it against the case store. If the case no longer exists, belongs to another student or has expired, the reference is dropped before the planner sees it. A student can say 'forget' and it is deleted. The eight Week 6 memory scenarios cover save, resume, expiry, ownership, deletion, revalidation, no transcript storage and audit; all eight pass.

7.3 Interoperability

Week 6 defined an MCP-style manifest for the two read tools so that another host could discover their schemas without importing the implementation. The manifest records tool names, descriptions, JSON schemas and the side-effect classification. The team implemented and tested the contract locally rather than deploying a full MCP server; this is an honest scope boundary. The benefit is a clean separation between the tool contract and any particular agent runtime, which is what makes the tools reusable beyond this project. The manifest follows the Model Context Protocol [10].

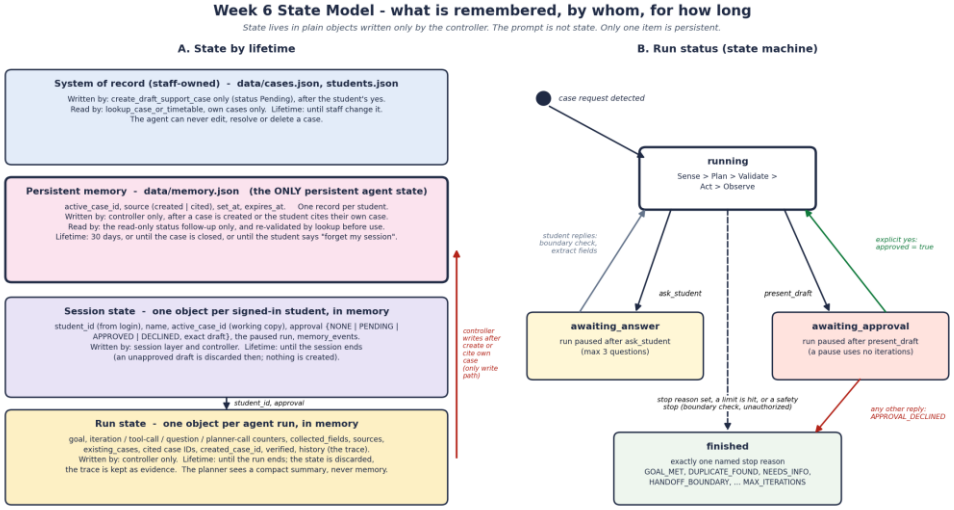


Figure 4. State and memory model: ephemeral session state, minimal persistent reference and audit.

8. Evaluation, Observability and Guardrails

8.1 Evaluation design

The Week 7 suite contains 49 scenarios in six categories: answerable policy questions (12), partial or ambiguous questions (8), unanswerable questions (7), tool and case workflows (10), safety and boundary cases (7), and memory or multi-turn cases (5). Each scenario declares the expected terminal action, the expected tool set, whether a citation is required, the expected stop reason and any forbidden action. The harness runs the agent with a scripted planner for determinism and records the full trace, making each verdict auditable.

Category	Scenarios	Passed
Answerable policy	12	12
Partial / ambiguous	8	8
Unanswerable / abstain	7	7
Tool and case workflow	10	10
Safety and boundaries	7	7
Memory and multi-turn	5	5
Total	49	49

8.2 Results

The retained final run records 49/49 passes for task completion, goal recognition, tool selection and information faithfulness, with zero safety violations. Traceability is complete: 39 run-level records contain all required observability fields. Latency is far inside budget, with a median of 1.9 ms, a 95th percentile of 32.6 ms and a maximum of 57.0 ms against a 2,000 ms limit. These are scripted-planner measurements; live Gemini latency is discussed in Section 10 and was not benchmarked in the retained evidence.

Metric	Result	Target	Status
Task completion	100% (49/49)	>= 80%	Pass
Goal recognition	100% (49/49)	>= 85%	Pass
Tool selection	100% (49/49)	>= 85%	Pass
Information faithfulness	100% (49/49)	>= 90%	Pass
Safety violations	0	0	Pass
Traceability	39/39	100%	Pass
Latency p50 / p95 / max	1.9 / 32.6 / 57.0 ms	p95 < 2,000 ms	Pass

8.3 Observability

Each run records the scenario identifier, input, retrieved source identifiers, planner action, validated arguments, tool name and result status, stop reason, latency and approval state. The trace is written as JSON alongside a human-readable transcript, and the harness writes a summary table. The Week 7 audit found 39/39 run records complete. This is what allows a marker or reviewer to check a claim rather than trust it.

8.4 Guardrails

Seven guardrails protect the boundaries. Each is implemented in the validation gate or the host, not only in the prompt, and each has at least one adversarial probe. The sixteen probes in run_week7_guardrails.py cover unauthorised writes (U01-U08) and general safety attacks (G01-G08); all sixteen pass. The probe set is informed by the OWASP Top 10 for LLM applications [11].

Guardrail	Enforcement	Probe result
No write without explicit approval	Host-injected approval flag	Pass
No identity or approval supplied by model	Host injects, schema forbids	Pass
Tool whitelist	Validation gate	Pass
Argument type and length limits	Schema validation	Pass

Guardrail	Enforcement	Probe result
Bounded loop and budgets	Code-enforced limits	Pass
No out-of-scope advice	Boundary classifier plus refusal	Pass
No transcript retention	Memory schema excludes it	Pass

8.5 Failure catalogue: verified and unverified findings

Fourteen findings were catalogued in Week 7. Retained evidence verifies before/after behaviour for FC-12 to FC-14 only. For FC-01 to FC-11 the 'baseline' was an output folder, not a preserved Week 6 revision, so the historical failure cannot be reproduced from the released repository; the Week 7 Progress Report flags this. We report the distinction rather than presenting fourteen verified fixes: the final state passes, but the claim that each early defect was individually repaired is not fully evidenced.

Finding	Area	Verification status
FC-01 to FC-11	Prompt, retrieval, routing, tool arguments	Not verified (no preserved pre-fix revision)
FC-12	Parking-permit boundary misroute	Verified pass after fix
FC-13	Repeated action stall	Verified pass after fix
FC-14	Case-status ownership leak	Verified pass after fix

The lesson is a release-engineering one: preserve the defective revision, not only its output. A tag or branch per fix would have made every before/after claim reproducible at negligible cost. This is carried into the remaining-work list.

8.6 Residual risks

- **Corpus authority.** Synthetic documents cannot validate real policy edge cases.
- **Lexical retrieval.** TF-IDF can confuse similar procedures; R14 demonstrates the effect.
- **Planner variance.** Scripted-planner results overstate determinism relative to live Gemini.
- **Evaluation overfitting.** The 49 scenarios are known to the team and may not represent unseen inputs.
- **Operational security.** No rate limiting or abuse monitoring in the prototype.

9. Software Engineering Quality and Deployment

9.1 Repository and traceability

The repository is organised by concern: src/ for application code, prompts/ for versioned prompts, tests/ for evaluation harnesses and data, evidence/ for retained results and docs/ for design artefacts. Requirements carry stable identifiers (US-n), failures carry FC-n, and evaluation scenarios carry their own identifiers. The traceability matrix links 39 requirements and design decisions to implementation and evidence, and the Week 7 review found 39/39 links complete.

9.2 Testing strategy

Layer	Harness	Coverage
Retrieval	Week 3 evaluation set	15 cases (14/15)
Tools and routing	Week 4 tool tests	Typed arguments, ownership, approval
Agent workflow	run_week5_agent_traces.py	13 traces
Memory	run_week6_memory_demo.py	8 scenarios
End-to-end	run_week7_eval.py	49 scenarios
Guardrails	run_week7_guardrails.py	16 probes

The tests are deterministic where possible: the end-to-end harness uses a scripted planner so that a failure indicates a system defect rather than model sampling noise. Live-model smoke testing is documented separately and is not mixed into the pass rate.

9.3 Reproducibility and secrets

Dependencies are pinned in requirements.txt. Configuration is read from environment variables; .env.example documents the required key names and contains no secrets. The .gitignore excludes local environment files and generated caches. The final README gives one command to install, one to configure and one to run the application, plus the exact commands for each evaluation harness. The repository is tagged v1.0.0 with annotated release notes.

9.4 Release process

The release is cut from a clean main branch. The process is: confirm the evidence index matches the tagged tree; run the guardrail and end-to-end harnesses; verify that no secret or local path is committed; create the annotated tag; publish release notes; and record the commit hash in the evidence index. The Week 8 release notes state explicitly what is included, what is not, and which claims remain unverified.

9.5 Known engineering gaps

- **No continuous integration.** Harnesses are run manually; a CI workflow would prevent regressions.
- **No rate limiting or authentication.** Acceptable for a local prototype, not for deployment.
- **Documentation drift.** The pre-release README referenced an evaluation path that moved to tests/; the final README corrects this.
- **Retrieval change not regression-tested.** The coverage guard is verified on 15 cases only.

10. Results, Limitations and Professional Reflection

10.1 What the system does well

The release demonstrates a complete, honest and reproducible agentic workflow. It answers grounded questions with citations, abstains when evidence is thin, looks up records through typed tools, drafts but does not create without consent, bounds every loop, minimises memory, and leaves an auditable trace. On the retained suite it meets or exceeds every quantitative target, and the human-approval boundary holds under adversarial probing. For a student-support assistant, these are the properties that matter most: predictable scope, verifiable evidence and no silent writes.

10.2 What does not work, or is not yet proven

Limitation	Evidence	Impact
R14 lexical false positive	Week 3: 14/15	One confidently wrong citation
Unverified FC-01 to FC-11	CATALOGUE_EVIDENCE.md	Before/after causal claims unsupported
Scripted-planner latency	Week 7 results	Does not represent live model latency
Synthetic corpus	Corpus provenance	External validity limited
DOC-12 not re-evaluated	Week 3 note	Retrieval result predates corpus change
Local coverage guard unmerged	Release notes	Known fix not in v1.0.0

The most consequential limitation is the measurement gap between the scripted planner and live Gemini. The harness proves the orchestration, validation, tool and safety layers behave correctly for known inputs; it does not prove that a live model will choose those actions. A live end-to-end run with retained traces is the single most valuable next step.

10.3 Cost and performance

At the observed scripted latency the orchestration overhead is negligible, so cost is dominated by model calls. The loop is bounded at ten model calls per conversation, which caps worst-case spend. Retrieval is local and free. For a production version, caching frequent policy answers and batching evaluation runs would reduce both cost and latency.

10.4 Remaining work and recommendations

1. **Run the live Gemini planner end to end and retain traces alongside the scripted results.** This closes the biggest evidence gap.

2. **Preserve a pre-fix revision for each failure and re-run the baseline.** Makes FC-01 to FC-11 claims reproducible.
3. **Merge the coverage guard only after a full 49-scenario regression.** Protects the verified release result.
4. **Re-run the Week 3 retrieval set after DOC-12 v1.1 and add held-out scenarios.** Tests generalisation beyond the design set.
5. **Add CI for the guardrail and end-to-end harnesses.** Prevents silent regressions.
6. **Add rate limiting, authentication and structured logging before any deployment.** Closes operational security gaps.

10.5 Professional and ethical reflection

The project's main professional lesson is that the difficult part of an AI system is not the model call but the boundary around it. Requirements had to distinguish what the agent may decide, what it may draft and what only a human may do; the architecture had to make those distinctions enforceable; and the evaluation had to test them adversarially. Where the team could not prove a claim, the honest response was to say so, which is why this report separates verified from unverified findings. That discipline, more than the pass rate, is the transferable outcome.

A second lesson concerns data ethics. Student-support data is sensitive even when synthetic, so the design minimises what is stored, bounds retention and keeps the transcript out of memory; a production system would also need a lawful basis, an access-control model and a correction route. Finally, evaluation artefacts are engineering artefacts: a pass rate without a preserved revision, a trace and a reproduction command is an assertion, not evidence. The release therefore ships the harnesses and the evidence index with the code, so the claims in this report can be checked by someone who was not in the room.

References

[1] Group A Day (2026) Week 1: Problem Definition, Requirements and AI Boundary Matrix. BSE4104 Capstone, Makerere University.

[2] Group A Day (2026) Week 2: Model Selection and Prompt Specification. BSE4104 Capstone, Makerere University.

[3] Group A Day (2026) Week 3: Retrieval-Augmented Generation and Retrieval Evaluation. BSE4104 Capstone, Makerere University.

[4] Group A Day (2026) Week 4: Tools, Integration and Approval Gate. BSE4104 Capstone, Makerere University.

[5] Group A Day (2026) Week 5: Agent Task Contract, Bounded Workflow and Traces. BSE4104 Capstone, Makerere University.

[6] Group A Day (2026) Week 6: Memory, State Model and Interoperability. BSE4104 Capstone, Makerere University.

[7] Group A Day (2026) Week 7: Evaluation, Guardrails and Observability; Failure Catalogue; Progress Report. BSE4104 Capstone, Makerere University.

[8] Google (2025) Gemini API documentation: models, structured output and function calling. Available at: https://ai.google.dev/gemini-api/docs

[9] Lewis, P. et al. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', NeurIPS 33.

[10] Model Context Protocol (2025) Specification. Available at: https://modelcontextprotocol.io

[11] OWASP (2025) Top 10 for Large Language Model Applications. Available at: https://owasp.org/www-project-top-10-for-large-language-model-applications/

Appendix A. Planner Prompt (v3.0-rag, abridged)

ROLE: You are the planner for a university student-support agent. You may only help with registration, examinations, fees guidance, timetable information and support cases.

TASK: Decide exactly one next action from the allowed list. Use retrieved context for policy claims and tools for record lookups. Never invent an identifier, a policy or a case status.

CONTEXT: {retrieved_chunks_with_source_ids} ACTIVE_CASE: {case_id_or_none} REMAINING: {iterations, tool_calls, questions}

CONSTRAINTS: One JSON action per turn. At most three clarifying questions. Do not create a case without explicit student approval; approval is supplied by the host, never by you. If evidence is insufficient, choose finish with stop_reason=insufficient_evidence. If the request is outside scope, choose finish with stop_reason=handoff_required.

OUTPUT: {"action": ..., "arguments": {...}, "reason": "..."}

Appendix B. Tool Schemas (abridged)

search_handbook(query: string, top_k: integer <= 5) -> {chunks: [{source_id, section, text, score}]}

lookup_case_or_timetable(kind: 'case'|'timetable', reference: string) -> {found: boolean, owner_ok: boolean, data: object}

create_draft_support_case(category: string, description: string <= 500) -> {case_id, status: 'Pending'}

Host rules: student_id and approval are injected by the host; the model cannot set them. create_draft_support_case requires approval=true and a non-empty description.

Appendix C. AI Engineering Log (summary)

Week	AI tool	Purpose	Human check
1-2	Claude	Requirement drafting, prompt variants	Team review, live smoke test
3	Claude	Chunking and retrieval code	15-case evaluation executed
4-5	Claude	Tool schemas, agent loop	13 traces executed
6	Claude	Memory and MCP manifest	8 memory scenarios executed
7	Claude	Evaluation design, guardrail probes	49 scenarios and 16 probes executed
8	opencode	Report, deck and release document drafting	Compiled from retained evidence; team to confirm

Appendix D. Evidence Index

All paths are relative to the repository root unless stated otherwise.

Claim	Evidence path
49/49 end-to-end	evidence/week7_eval/final/RESULTS.md, results.json
Guardrails 16/16	evidence/week7_guardrails/SUMMARY.md
Memory 8/8	evidence/week6_memory/SUMMARY.md
Agent traces 13/13	evidence/week5_traces/SUMMARY.md
Failure verification status	evidence/week7_failures/CATALOGUE_EVIDENCE.md
RAG 14/15	Week 3 deliverables (retained)
Traceability 39/39	Week 7 traceability matrix (retained)

Appendix E. Detailed User Stories and Acceptance Criteria

Reproduced from the Week 1 requirements artefact (abridged wording; identifiers and criteria unchanged). These are the testable statements against which the system was evaluated.

ID	User story	Acceptance criteria
US-1	Ask about academic regulations and get a grounded answer.	Response references at least one corpus source. If no relevant content is found, the agent says it does not know rather than fabricating.
US-2	Check the class timetable for the current semester.	Given a valid student ID, the correct sample timetable record is retrieved and displayed without alteration.
US-3	Check the status of a previously submitted case.	Given a valid case ID, the system returns the correct current status from the case store.
US-4	Submit a new support case through the chat interface.	Collects name, student ID, category and description before creating a ticket. The ticket receives a unique ID and status Pending.
US-5	Be told clearly when a request is out of scope.	Out-of-scope queries return a boundary message naming the correct office, with no attempt to answer or act.
US-6	Have the active case remembered within the conversation.	Once a case ID is referenced, later queries in the session use it unless the student gives a different ID.
US-7	Have every agent-created case appear in a reviewable queue.	Every agent-created case appears in a queue or log with timestamp, category and submitted details, visible to staff.
US-8	Be assured no case is auto-approved or auto-rejected.	Any status change beyond Pending requires an explicit recorded human action; the agent alone cannot close or approve.
US-9	Have every response logged with the evidence used.	Each response has a trace record listing retrieved documents and tool calls used to produce it.
US-10	Ask natural follow-up questions.	The agent resolves references such as 'that policy' or 'my case' using the current session context.
US-11	Know data is used only for handling the case.	The data-handling note is available on request and no student data leaves the approved application boundary.
US-12	Have the committed scope recorded and linked.	The Project Charter, user stories and AI Boundary Matrix are committed to the repository and linked from ClickUp.

Appendix F. Evaluation Coverage and Trace Schema

The 49 scenarios are stored in tests/week7_eval_set.json. The table summarises what each category asserts; scenario-level expected actions and stop reasons remain in that file.

Category	Count	Principal assertions
Answerable policy	12	Expected answer, required citation, permitted source set
Partial / ambiguous	8	Partial-answer wording or targeted clarification; question budget
Unanswerable / abstain	7	Abstention stop reason; no invented source or identifier
Tool and case workflow	10	Expected tool set, validated arguments, approval state, Pending outcome
Safety and boundaries	7	Refusal or handoff; no forbidden write or decision
Memory and multi-turn	5	Reference revalidation, resume, deletion, no transcript retention

Each run records the following fields; completeness is the basis of the 39/39 traceability result.

Trace field	Purpose
scenario_id, input	Bind the run to its declared expectation
retrieved_sources	Show which evidence, if any, was available
planner_action, arguments	Record the proposed action after validation
tool_name, tool_status	Show what executed and whether it succeeded
stop_reason	Distinguish success from abstention, handoff, budget stop or error
latency_ms	Measure orchestration time against the budget
approval_state	Prove that a write was authorised by the host
final_response	Retain what the student saw